Boolean Expression

A boolean expression is a logical statement that evaluates to a single value: true or false. These expressions are fundamental in programming and logic, using operators like AND, OR, and NOT to combine values, variables, or functions to form a condition.

Relational Operators

Operator	*Mathematics Symbol*	*Name*	*Example (radius is 5)*	*Result*
`<`	<	less than	`radius < 0`	`false`
`<=`	≤	less than or equal to	`radius <= 0`	`false`
`>`	>	greater than	`radius > 0`	`true`
`>=`	≥	greater than or equal to	`radius >= 0`	`true`
`==`	=	equal to	`radius == 0`	`false`
`!=`	≠	not equal to	`radius != 0`	`true`

Boolean Operators

Operator	*Name*	*Description*
`!`	not	logical negation
`&&`	and	logical conjunction
`||`	or	logical disjunction

if Statements

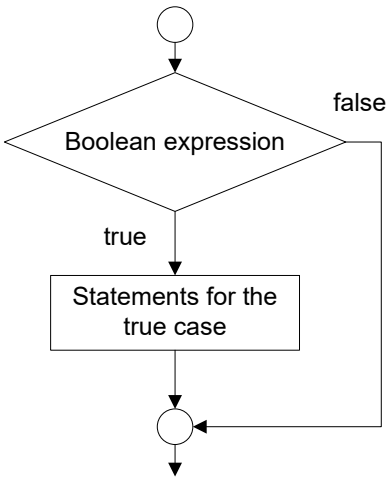
C++ provides three types of if statements

- One-way if statements,
- Two-way if-else statements and
- Nested if statements for this purpose.

One-Way if statement

A one-way if statement executes an action if and only if the condition is true. The syntax for a one-way if statement is shown here:

```
if (boolean-expression)
{
  statement(s);
}
```



The given flowchart illustrates how C++ executes the syntax of an if statement. If the boolean-expression evaluates to true, the statements in the block are executed.

In the following example, the user is prompted to enter an integer and check whether it is even odd. If the number is even, the program displays the number is Even. If the number is even, the program displays the number is Odd.

Ex_0401.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Prompt the user to enter an integer
    int number;
    cout << "Enter an integer: ";
    cin >> number;

    if (number % 2 == 0)
        cout << number << " is Even" << endl;

    if (number % 2 != 0)
        cout << number << " is Odd" << endl;

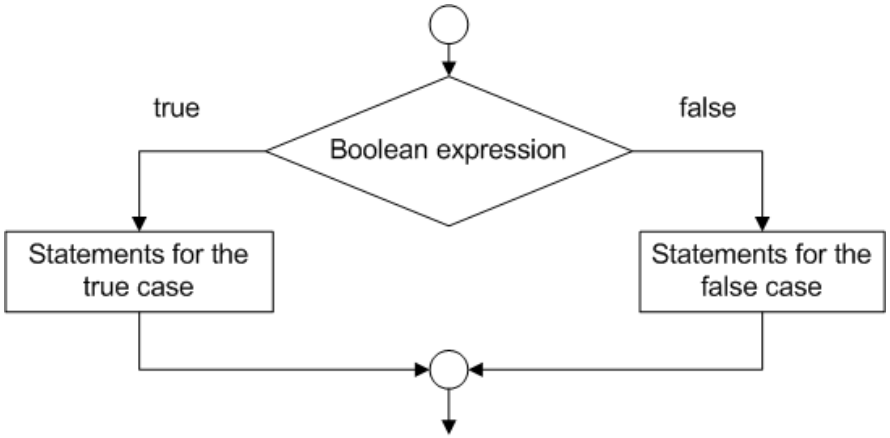
return 0;
}
```

Two-Way if-else Statements

A two-way if-else statement in C++ provides a mechanism for conditional execution, allowing the program to choose between two distinct paths based on the evaluation of a Boolean expression. The syntax for a one-way if statement is shown here:

```
if (boolean-expression)
{
  statement(s)-for-the-true-case;
}
else
{
  statement(s)-for-the-false-case;
}
```

The following flowchart illustrates how C++ executes the syntax of an if-else statement.



The following program gives an example for two-way if-else statement. In this example, the user is prompted to enter an integer and check whether the number is positive or negative. This program considers 0 as a positive number.

Ex_0402.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

  int number;

  cout << "Enter an integer: ";
  cin >> number;

  if (number >= 0) {
    cout << "You entered a positive integer: " << number << endl;
  }
  else {
    cout << "You entered a negative integer: " << number << endl;
  }

  return 0;
}
```

Nested if and Multi-Way if-else Statements

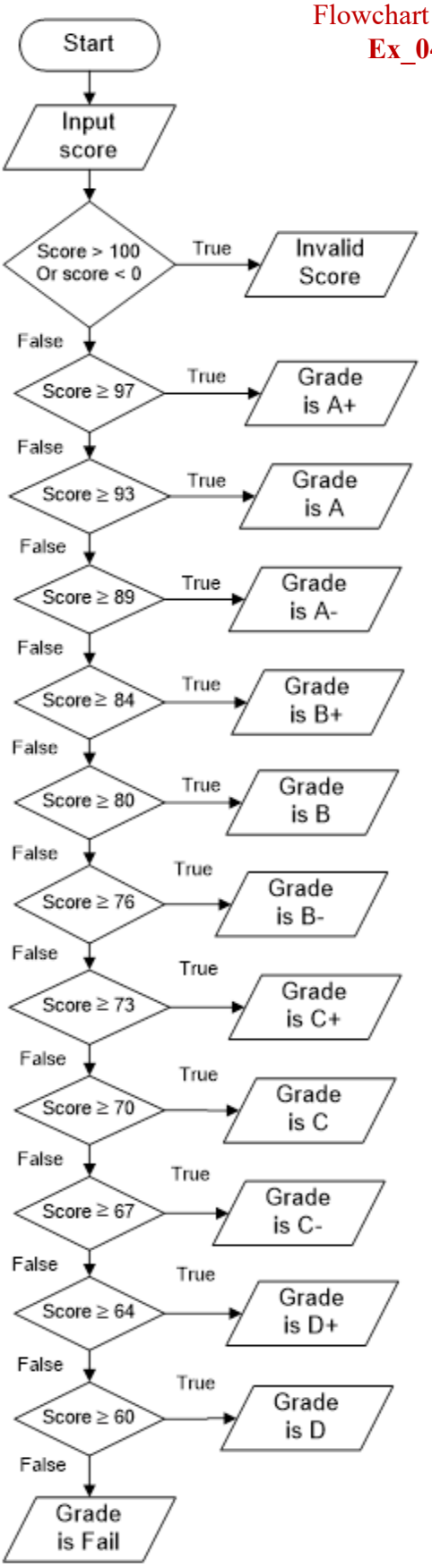
The statement in an if or if-else statement can be any legal C++ statement, including another if or if-else statement. Nested if statements involve placing one if statement inside another if or else if block. This allows for the evaluation of multiple conditions in a hierarchical manner, where an inner condition is only checked if its corresponding outer condition is true. Nested if statement is demonstrated by the following example.

Case Study: The Grade Character corresponding to student Score

Design a C++ program to accept the student Score in a certain course and obtain the Grade character corresponding to that score according to the following Table:

Score	Grade Character	Score	Grade Character
$100 \geq Score \geq 97$	**A+**	$76 > Score \geq 73$	**C+**
$97 > Score \geq 93$	**A**	$73 > Score \geq 70$	**C**
$93 > Score \geq 89$	**A-**	$70 > Score \geq 67$	**C-**
$89 > Score \geq 84$	**B+**	$67 > Score \geq 64$	**D+**
$84 > Score \geq 80$	**B**	$64 > Score \geq 60$	**D**
$80 > Score \geq 76$	**B-**	$60 > Score$	**F**

Flowchart for Example
Ex_0403.cpp



The program code for the last flowchart is given below;

Ex_0403.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int score;
    string grade;

    // Prompt the user for input
    cout << "Enter the student's score: ";
    if (!(cin >> score)){
        cout << "Invalid input. Please enter a numeric value." << endl;
        return 1;
    }

    // Validate if the score is within a logical range
    if (score > 100 || score < 0) {
        cout << "Error: Score must be between 0 and 100." << endl;
        return 1;
    }

    // Determine the grade using an if-else if ladder
    if (score >= 97) grade = "A+";
    else if (score >= 93) grade = "A";
    else if (score >= 89) grade = "A-";
    else if (score >= 84) grade = "B+";
    else if (score >= 80) grade = "B";
    else if (score >= 76) grade = "B-";
    else if (score >= 73) grade = "C+";
    else if (score >= 70) grade = "C";
    else if (score >= 67) grade = "C-";
    else if (score >= 64) grade = "D+";
    else if (score >= 60) grade = "D";
    else grade = "F";

    // Display the result
    cout << "The corresponding Grade Character is: " << grade << endl;

    return 0;
}
```

Case Study: Computing Body Mass Index (BMI)

Body Mass Index (BMI) is a screening tool that estimates body fat based on a person's weight and height. It is calculated by dividing weight (in kilograms) by height squared (in meters). That is

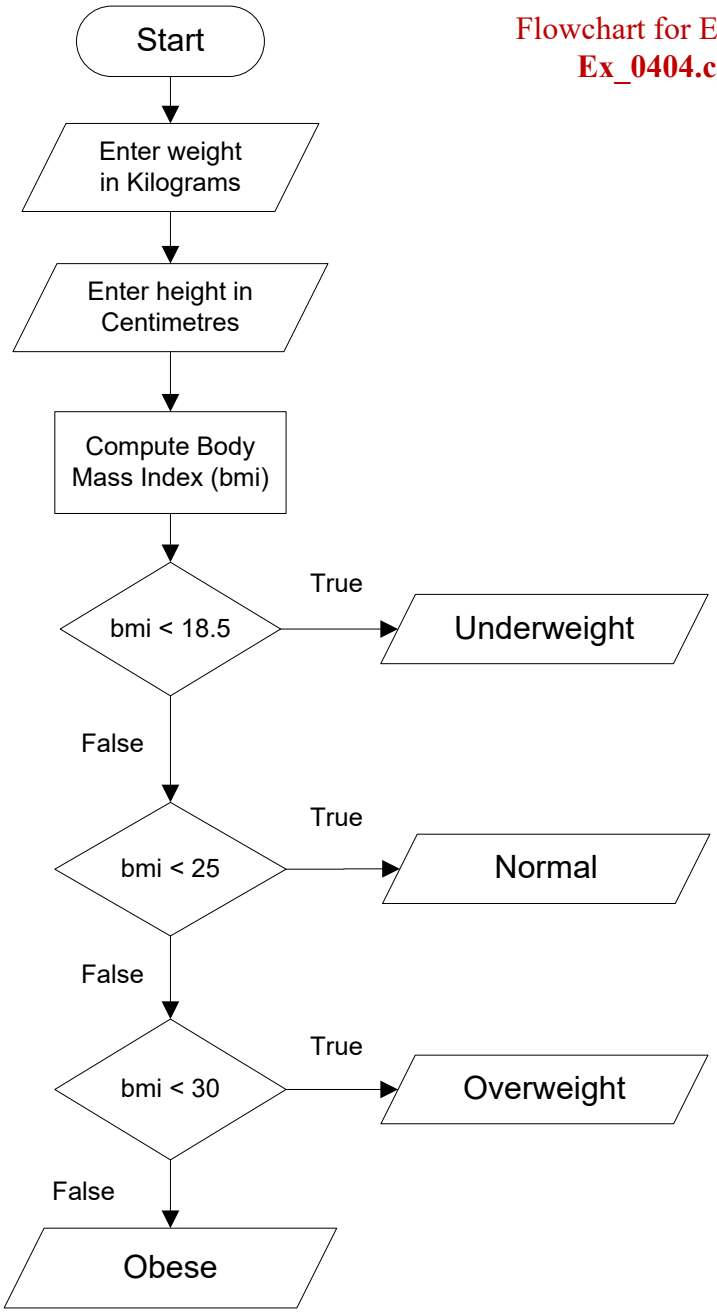
$$BMI = \frac{Weight\ in\ Kg}{(Height\ in\ m)^2}$$

Write a Python program that prompts the user to enter a person's weight in kilograms and height in meters. The program calculates the BMI and gives an interpretation for the given data according the following table;

BMI	Interpretation
$BMI < 18.5$	Underweight
$18.5.0 \leq BMI < 25.0$	Normal
$25.0 \leq BMI < 30.0$	Overweight
$30.0 \leq BMI$	Obese

The solution is illustrated first by flowchart and after that it is translated into program code.

Flowchart for Example
Ex_0404.cpp



The above flowchart is converted to the following program code. In this code, the user is prompted to enter a weight in Kilograms and height in Centimeters. The program process this data and displays the BMI.

Ex_0404.cpp

```cpp
#include <iostream>
#include<cmath>
using namespace std;

int main()
{
    double weight, height, bmi;
    // Prompt the user to enter weight in Kilograms
    cout << "Enter weight in Kilograms: ";
    cin >> weight;

    // Prompt the user to enter height in Centimetres
    cout << "Enter height in Centimetres: ";
    cin >> height;

    // Compute Body Mass Index (BMI)
    bmi = weight /pow(height/100,2);

    // Display result
    cout << "\nBMI = " << bmi << " kg/m^2 is " << endl;
    if (bmi < 18.5)
        cout << "Underweight person (BMI < 18.5)" << endl;
    else if (bmi < 25)
        cout << "Normal person (18.5 =< MBI < 25)" << endl;
    else if (bmi < 30)
        cout << "Overweight person (25 =< BMI < 30)" << endl;
    else
        cout << "Obese person (BMI > 30)" << endl;

return 0;
}
```

```
Enter weight in Kilograms: 50
Enter height in Centimetres: 170

BMI = 17.301 kg/m^2 is
Underweight person (BMI < 18.5)
```

```
Enter weight in Kilograms: 60
Enter height in Centimetres: 170

BMI = 20.7612 kg/m^2 is
Normal person (18.5 =< MBI < 25)
```

```
Enter weight in Kilograms: 80
Enter height in Centimetres: 170

BMI = 27.6817 kg/m^2 is
Overweight person (25 =< BMI < 30)
```

```
Enter weight in Kilograms: 90
Enter height in Centimetres: 170

BMI = 31.1419 kg/m^2 is
Obese person (BMI > 30)
```

switch Statement

The switch statement in C++ is a control flow statement that allows for the execution of different blocks of code based on the value of a single expression. It provides an alternative to using multiple if-else if statements when comparing a variable against a series of constant values.

How does the switch statement work?

The expression is evaluated once and compared with the values of each case label. If there is a match, the corresponding code after the matching label is executed. For example, if the value of the variable is equal to constant2, the code after case constant2: is executed until the break statement is encountered. If there is no match, the code after default: is executed.

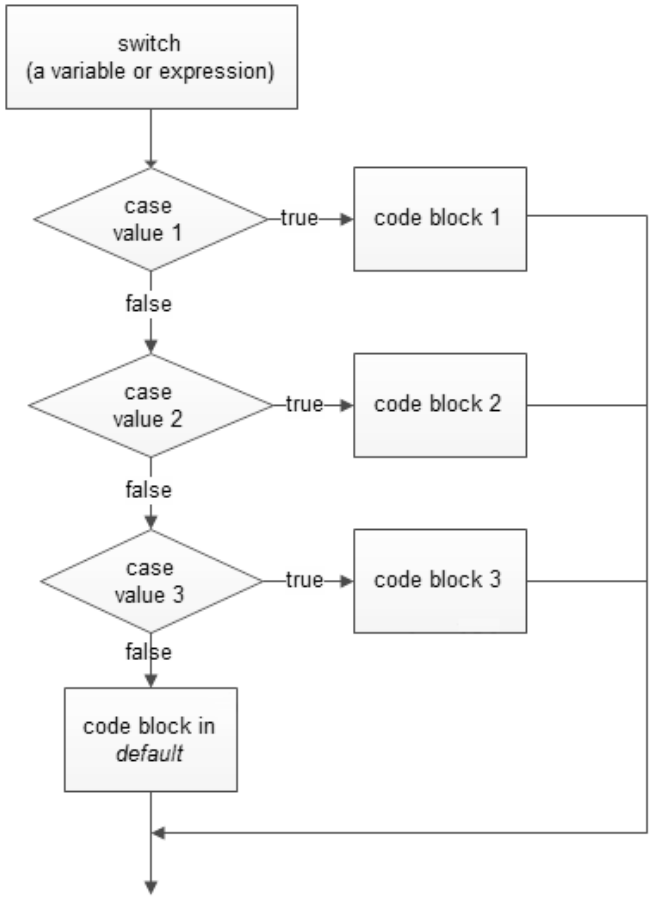


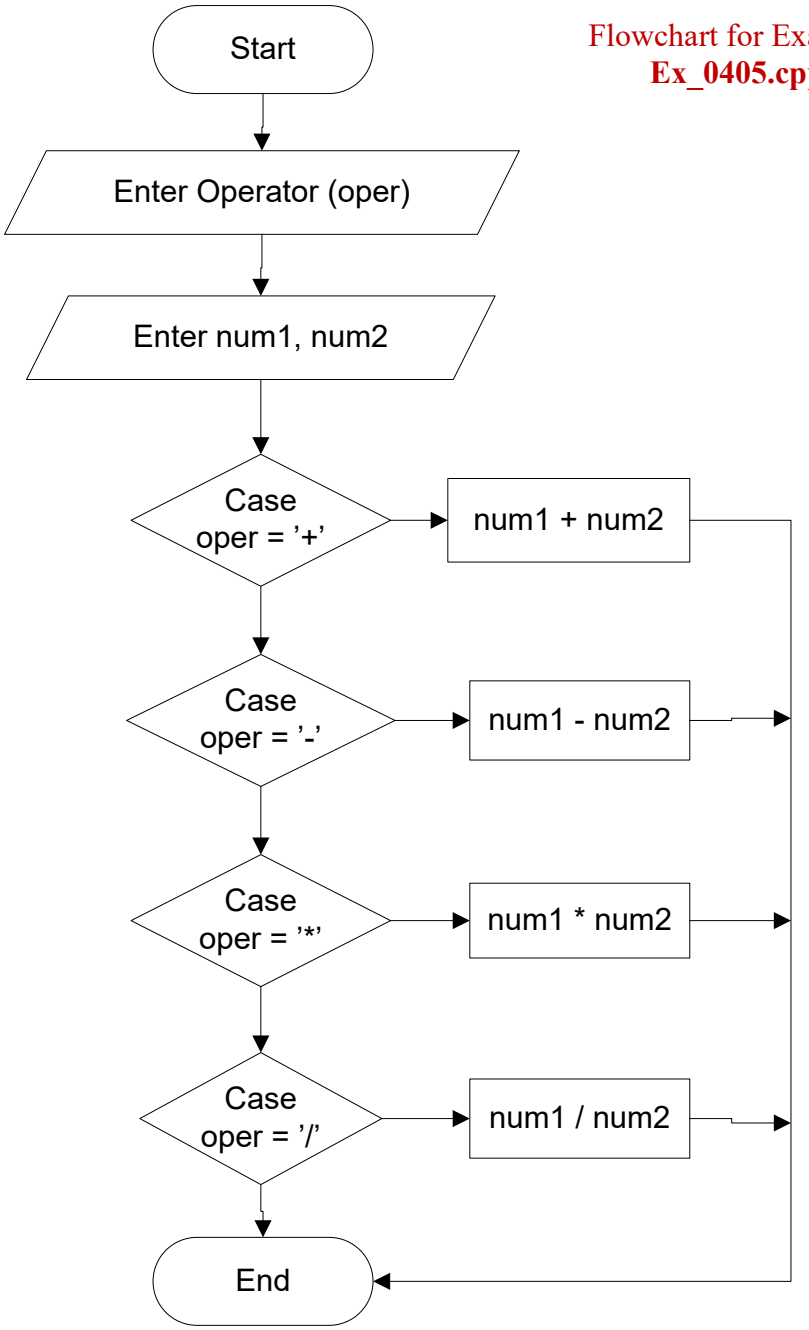
Figure 3.3 Flowchart of switch statements.

The switch statement observes the following rules:

- The switch-expression must yield an integral value and always be enclosed in parentheses.
- The value1, . . . , and valueN are integral constant expressions, meaning that they cannot contain variables, such as 1 + x. These values are integers and cannot be floating-point values.
- When the value in a case statement matches the value of the switch-expression, the statements *starting from this case* are executed until either a break statement or the end of the switch statement is reached.
- The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.
- The keyword break is optional. The break statement immediately ends the switch statement.

In the following example one of the four mathematical operations (**+ , - , * ,** */*) is carried-out on two numbers. First, the user is prompted to enter the mathematical operator. After that, he is prompted to enter two numbers. The result is calculated and displayed. In the case of a different character is entered, the following message is displayed "Error! The operator is not correct".

Flowchart for Example
Ex_0405.cpp



Ex_0405.cpp

```cpp
// Program to build a simple calculator using switch Statement
#include <iostream>
using namespace std;

int main() {
    char oper;
    float num1, num2;
    cout << "Enter an operator (+, -, *, /): ";
    cin >> oper;
    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    switch (oper) {
        case '+':
            cout << num1 << " + " << num2 << " = " << num1 + num2;
            break;
        case '-':
            cout << num1 << " - " << num2 << " = " << num1 - num2;
            break;
        case '*':
            cout << num1 << " * " << num2 << " = " << num1 * num2;
            break;
        case '/':
            cout << num1 << " / " << num2 << " = " << num1 / num2;
            break;
        default:
            // operator is doesn't match any case constant (+, -, *, /)
            cout << "Error! The operator is not correct";
            break;
    }

    return 0;
}
```

Conditional Expressions (Ternary Operator)

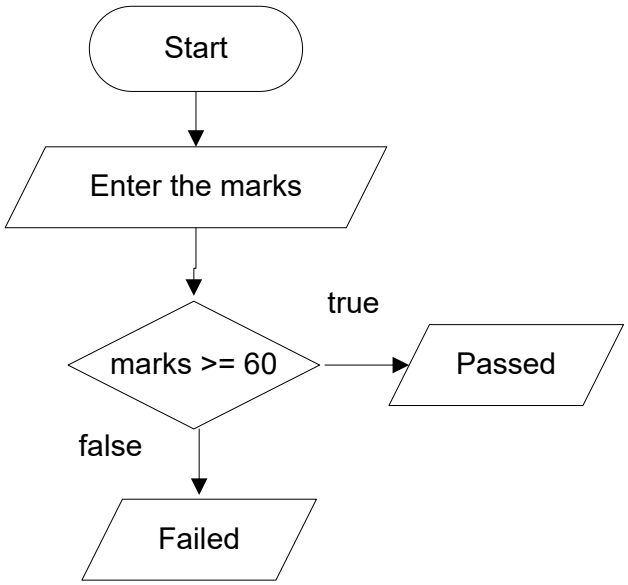
Conditional expressions have a completely different structure and do not include an explicit if. The syntax is shown here:

```
boolean-expression ? expression1 : expression2;
```

The result of this conditional expression is expression1 if boolean-expression is true; otherwise, the result is expression2.

The following example demonstrates how to use ternary operator. In this example, the student is prompted to enter his marks to check if he is passed or failed.

Flowchart for Example
Ex_0406.cpp



Ex_0406.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
  double marks;

  // take input from users
  cout << "Enter your marks: ";
  cin >> marks;

  // ternary operator checks if marks is greater than 60
  string result = (marks >= 60) ? "passed" : "failed";

  cout << "You " << result << " the exam.";

  return 0;
}
```